# Explicación de `modelo.py`

Un robot que aprende solito a salir de un laberinto, explicado línea por línea.

> **La idea en pocas palabras:** imagina un ratón metido en un laberinto. Al principio no sabe nada y se mueve casi al azar. Cada vez que hace algo bueno (acercarse a la salida o llegar) le damos puntos; cuando hace algo malo (chocar o alejarse) se los quitamos. El ratón va apuntando en una "libreta" qué tan buena fue cada jugada en cada casilla. Después de muchos intentos, la libreta le dice cuál es el mejor movimiento en cada lugar, y ya sabe llegar a la meta. A esta forma de aprender se le llama **Q-learning**, y a la libreta se le llama **tabla Q**.

## 1. Herramientas

```
1  import numpy as np
```

Trae una librería llamada **numpy** (y le pone el apodo `np`). Sirve para manejar tablas de números de forma fácil y para sacar números al azar. La vamos a usar para la "libreta" del ratón.

## 2. El laberinto

```
3  #significa
4  # '#' = pared   '.' = libre   'S' = inicio   'M' = meta
5  LABERINTO = [
6      "S..#....",
7      ".#.#.##.",
       ...
13     "#...#..M",
14 ]
```

- **Líneas 3–4:** son comentarios (empiezan con `#`), Python no los ejecuta. Solo explican qué significa cada símbolo.
- **Línea 5:** crea el laberinto y lo guarda con el nombre `LABERINTO`. Está en mayúsculas porque es algo que no va a cambiar.
- **Líneas 6–13:** cada texto entre comillas es un renglón del laberinto. Hay 8 renglones de 8 casillas, o sea un cuadro de 8×8. La `S` (salida/inicio) está arriba a la izquierda y la `M` (meta) abajo a la derecha.
- **Línea 14:** cierra la lista.

Para ubicar una casilla se usan dos números: **fila** (de arriba hacia abajo, empezando en 0) y **columna** (de izquierda a derecha, empezando en 0). La S está en (0, 0) y la M en (7, 7).

## 3. Los movimientos

```
18 ACCIONES = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # arriba, abajo, izquierda, derecha
19 FLECHAS = ["↑", "↓", "←", "→"]
```

- **Línea 18:** los 4 movimientos posibles. Cada par dice cuánto cambia la fila y cuánto cambia la columna:

| Número | Par | Qué hace |
|---|---|---|
| 0 | (-1, 0) | sube una fila → **arriba** |
| 1 | (1, 0) | baja una fila → **abajo** |
| 2 | (0, -1) | una columna a la izquierda → **izquierda** |
| 3 | (0, 1) | una columna a la derecha → **derecha** |

  En el resto del programa cada movimiento se nombra con su número (0, 1, 2 o 3).
- **Línea 19:** el dibujito de flecha de cada movimiento, en el mismo orden. Sirve para mostrar el resultado de forma bonita (en este archivo no se usa, seguramente lo usa otro archivo que dibuja el laberinto).

## 4. Premios y castigos

```
22 PREMIO_META = 100
23 CASTIGO_CHOQUE = -10
24 PREMIO_ACERCARSE = 1      # gana 1 punto si se acerca a la meta
25 CASTIGO_ALEJARSE = -2     # pierde el doble si se aleja
26 MAX_PASOS = 40  # si no llega en 40 pasos, el intento falla
```

- **Línea 22:** llegar a la meta da **+100** puntos. Es el gran premio.
- **Línea 23:** chocar contra una pared (o intentar salirse del mapa) quita **10** puntos.
- **Línea 24:** dar un paso que lo acerca a la meta da **+1**.
- **Línea 25:** dar un paso que lo aleja quita **2**. Se castiga más alejarse que lo que se premia acercarse, para que no ande dando vueltas.
- **Línea 26:** cada intento tiene como máximo **40 pasos**. Si no llega en ese tiempo, ese intento cuenta como fallido.

## 5. Función `buscar`: encontrar una letra

```
28 def buscar(laberinto, letra):
29     for f, renglon in enumerate(laberinto):
30         for c, celda in enumerate(renglon):
31             if celda == letra:
32                 return (f, c)
```

- **Línea 28:** `def` crea una función (una "receta" que se puede usar muchas veces). Esta recibe el laberinto y una letra, por ejemplo `"S"` o `"M"`.
- **Línea 29:** recorre los renglones uno por uno. `f` es el número de fila y `renglon` el texto de ese renglón.
- **Línea 30:** dentro de cada renglón, recorre casilla por casilla. `c` es el número de columna y `celda` el símbolo.
- **Línea 31:** pregunta: ¿esta casilla es la letra que busco?
- **Línea 32:** si sí, devuelve su posición (fila, columna) y termina. Ejemplo: `buscar(LABERINTO, "M")` da `(7, 7)`.

## 6. Función `es_pared`: ¿puedo pisar ahí?

```
35 def es_pared(laberinto, f, c):
36     filas, cols = len(laberinto), len(laberinto[0])
37     if f < 0 or f >= filas or c < 0 or c >= cols:
38         return True  # salirse del mapa cuenta como pared
39     return laberinto[f][c] == "#"
```

- **Línea 35:** función que recibe una posición (fila `f`, columna `c`) y responde **Sí** (`True`) o **No** (`False`).
- **Línea 36:** cuenta cuántas filas tiene el laberinto (8) y cuántas columnas tiene el primer renglón (8).
- **Línea 37:** revisa si la posición está fuera del mapa: fila negativa, fila demasiado grande, columna negativa o columna demasiado grande.
- **Línea 38:** si está fuera, responde "sí es pared". Así el ratón no puede escaparse del mapa.
- **Línea 39:** si está dentro, responde "sí" solo cuando en esa casilla hay un `#`.

## 7. Función `distancia`: ¿qué tan lejos estoy?

```
42 def distancia(a, b):
43     # cuantas filas + cuantas columnas faltan para llegar
44     return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

- **Línea 42:** recibe dos posiciones, `a` y `b`.
- **Línea 44:** resta las filas, resta las columnas, les quita el signo menos (`abs`) y las suma. Es como contar cuadras en una ciudad: "me faltan 3 calles hacia abajo y 2 hacia la derecha = 5". No toma en cuenta las paredes, es solo una idea aproximada de la distancia.

Ejemplo: de (0, 0) a (7, 7) → 7 + 7 = **14**.

## 8. Función `paso`: dar un movimiento

Es el "juez" del juego: el ratón dice qué movimiento quiere hacer y esta función le dice dónde quedó y cuántos puntos gana o pierde.

```
47 def paso(laberinto, estado, accion):
48     df, dc = ACCIONES[accion]
49     nf, nc = estado[0] + df, estado[1] + dc
```

- **Línea 47:** recibe el laberinto, el **estado** (la casilla donde está el ratón ahora) y la **acción** (número del movimiento, 0 a 3).
- **Línea 48:** busca en la lista `ACCIONES` cuánto cambia la fila (`df`) y la columna (`dc`).
- **Línea 49:** calcula la casilla nueva (`nf`, `nc`) sumando ese cambio a la posición actual.

```
51     if es_pared(laberinto, nf, nc):
52         return estado, CASTIGO_CHOQUE, False, True
53     if laberinto[nf][nc] == "M":
54         return (nf, nc), PREMIO_META, True, False
```

La función siempre devuelve 4 cosas: **(dónde quedó, puntos, ¿llegó a la meta?, ¿chocó?)**.

- **Líneas 51–52:** si la casilla nueva es pared, el ratón **se queda donde estaba**, pierde 10 puntos, no llegó a la meta y sí chocó.
- **Líneas 53–54:** si la casilla nueva es la `M`, se mueve ahí, gana 100 puntos, sí llegó y no chocó.

```
56     #gana si se acerca a la meta, pierde el doble si se aleja
57     meta = buscar(laberinto, "M")
58     if distancia((nf, nc), meta) < distancia(estado, meta):
59         return (nf, nc), PREMIO_ACERCARSE, False, False
60     return (nf, nc), CASTIGO_ALEJARSE, False, False
```

- **Línea 57:** si llegó hasta aquí es porque se movió a una casilla libre normal. Busca dónde está la meta.
- **Línea 58:** compara: ¿la distancia a la meta desde la casilla nueva es menor que desde la casilla anterior?
- **Línea 59:** si sí (se acercó), gana 1 punto.
- **Línea 60:** si no (se alejó), pierde 2 puntos.

## 9. Función `elegir_accion`: ¿qué hago ahora?

```
62 def elegir_accion(Q, estado, epsilon, rng):
63     if rng.random() < epsilon:
64         return int(rng.integers(4))
65     valores = Q[estado[0], estado[1]]
66     mejores = np.flatnonzero(valores == valores.max())
67     return int(rng.choice(mejores))
```

- **Línea 62:** recibe la libreta `Q`, la casilla actual, un número `epsilon` (que vale 0.1, o sea 10%) y `rng`, que es el "dado" para sacar números al azar.
- **Línea 63:** tira el dado: saca un número entre 0 y 1. Si sale menor que 0.1 (pasa más o menos 1 de cada 10 veces)...
- **Línea 64:** ...elige un movimiento **totalmente al azar** (0, 1, 2 o 3). Esto se llama **explorar**: probar cosas nuevas por si descubre un camino mejor.
- **Línea 65:** el otro 90% de las veces, mira en la libreta los 4 puntajes que tiene apuntados para esta casilla (uno por cada dirección).
- **Línea 66:** busca cuál o cuáles direcciones tienen el puntaje más alto. Puede haber empate (por ejemplo al principio, cuando todos valen 0).
- **Línea 67:** si hay empate, escoge una de las mejores al azar. Esto se llama **aprovechar**: usar lo que ya aprendió.

## 10. Función `entrenar`: aquí aprende el ratón

```
70 def entrenar(episodios=300, alpha=0.1, gamma=0.9, epsilon=0.1, semilla=0):
```

Crea la función de entrenamiento con valores ya puestos por defecto:

- **episodios = 300**: el ratón va a intentar salir del laberinto 300 veces.
- **alpha = 0.1**: qué tanto cambia su opinión con cada experiencia nueva. 0.1 = cambia poquito cada vez (aprende despacio pero seguro).
- **gamma = 0.9**: qué tanto le importa el futuro. 0.9 = le importa casi tanto lo que viene después como el premio inmediato. Así aprende que vale la pena caminar varios pasos para llegar a los 100 puntos.
- **epsilon = 0.1**: 10% de las veces se mueve al azar (explora).
- **semilla = 0**: hace que el "azar" sea siempre el mismo cada vez que corres el programa, para que los resultados se puedan repetir.

```
    rng = np.random.default_rng(semilla)
    Q = np.zeros((8, 8, 4))
    inicio = buscar(LABERINTO, "S")
    exitos = []
    puntos = []
```

- **Línea 71:** crea el "dado" de números al azar usando la semilla.
- **Línea 72:** crea la **libreta Q** llena de ceros. Tiene 8 × 8 × 4: para cada una de las 64 casillas, guarda 4 números, uno por cada dirección. Cada número significa "qué tan buena idea es moverme hacia allá desde aquí". Al principio todo vale 0 porque no sabe nada.
- **Línea 73:** busca dónde está la `S` para saber desde dónde arranca.
- **Línea 74:** lista vacía para anotar en cada intento si llegó (1) o no (0).
- **Línea 75:** lista vacía para anotar cuántos puntos juntó en cada intento.

```
    for _ in range(episodios):
        estado = inicio
        llego = False
        total = 0
```

- **Línea 77:** repite todo lo de adentro 300 veces (un intento cada vez). El `_` significa "no me importa el número de vuelta".
- **Línea 78:** al empezar cada intento, el ratón vuelve a la casilla de inicio.
- **Línea 79:** todavía no ha llegado a la meta.
- **Línea 80:** su contador de puntos empieza en 0.

```
        for _ in range(MAX_PASOS):
            accion = elegir_accion(Q, estado, epsilon, rng)
            nuevo, r, termino, choco = paso(LABERINTO, estado, accion)
            total += r
```

- **Línea 81:** dentro de cada intento, da hasta 40 pasos.
- **Línea 82:** decide qué movimiento hacer (a veces al azar, casi siempre el mejor según su libreta).
- **Línea 83:** hace el movimiento y el juez le responde: `nuevo` = dónde quedó, `r` = puntos que ganó o perdió, `termino` = si llegó a la meta, `choco` = si chocó (aquí no se usa).
- **Línea 84:** suma esos puntos a su total del intento.

```
86              # FORMULA DE Q-LEARNING
87              futuro = 0 if termino else Q[nuevo[0], nuevo[1]].max()
88              Q[estado[0], estado[1], accion] += alpha * (r + gamma * futuro -
Q[estado[0], estado[1], accion])
```

Estas dos líneas son **el corazón del aprendizaje**. Aquí el ratón actualiza su libreta.

- **Línea 87:** calcula cuánto "promete" la casilla a la que llegó: mira en la libreta el mejor puntaje de esa casilla nueva. Si ya llegó a la meta, el futuro vale 0 porque ahí se acaba el juego.
- **Línea 88:** corrige el puntaje del movimiento que acaba de hacer. En palabras:

> **puntaje nuevo = puntaje viejo + 0.1 × (lo que realmente pasó − lo que yo creía)**
>
> donde "*lo que realmente pasó*" = puntos que recibió + 0.9 × lo mejor que puede hacer desde la casilla nueva.
>
> **Ejemplo:** el ratón está junto a la meta, su libreta dice que ir a la derecha vale 0. Va a la derecha y llega: gana 100.
> Lo que pasó = 100 + 0 = 100. Lo que creía = 0. Diferencia = 100.
> Puntaje nuevo = 0 + 0.1 × 100 = **10**. Ahora "ir a la derecha desde aquí" vale 10.
> La próxima vez que el ratón esté en la casilla anterior a esa, verá que desde ahí se puede conseguir algo bueno, y ese valor también sube. Así, poco a poco, el premio de la meta se va "contagiando" hacia atrás hasta llegar al inicio.

```
90              estado = nuevo
91              if termino:
92                  llego = True
93                  break
```

- **Línea 90:** ahora el ratón está en la casilla nueva.
- **Líneas 91–93:** si llegó a la meta, lo anota como logrado y `break` corta los pasos de este intento (ya no necesita seguir caminando).

```
94          exitos.append(int(llego))
95          puntos.append(total)
96
97      return Q, exitos, puntos
```

- **Línea 94:** al terminar cada intento, anota 1 si llegó o 0 si no (`int` convierte Verdadero/Falso en 1/0).
- **Línea 95:** anota los puntos totales de ese intento.
- **Línea 97:** al terminar los 300 intentos, entrega la libreta aprendida, la lista de éxitos y la lista de puntos.

## 11. Función `tasa_exito`: porcentaje de éxito

```
99  def tasa_exito(exitos, ventana=20):
100     tasa = []
101     for i in range(len(exitos)):
102         ultimos = exitos[max(0, i - ventana + 1): i + 1]
103         tasa.append(sum(ultimos) / len(ultimos) * 100)
104     return tasa
```

- **Línea 99:** recibe la lista de éxitos (unos y ceros) y una "ventana" de 20.
- **Línea 100:** lista vacía para guardar porcentajes.
- **Línea 101:** recorre cada intento, uno por uno.
- **Línea 102:** toma los últimos 20 intentos hasta el actual (si todavía no hay 20, toma los que haya; por eso el `max(0, ...)`, para no pedir posiciones negativas).
- **Línea 103:** suma cuántos fueron éxitos, divide entre cuántos intentos tomó y multiplica por 100 para tener un porcentaje. Ejemplo: 18 éxitos de 20 = 90%.
- **Línea 104:** entrega la lista de porcentajes. Sirve para ver cómo el ratón va mejorando con el tiempo (ideal para una gráfica).

## 12. Función `politica`: el mapa de flechas

```
107 def politica(Q):
108     flechas = {}
109     for f, renglon in enumerate(LABERINTO):
110         for c, celda in enumerate(renglon):
111             if celda in ".S":
112                 flechas[(f, c)] = int(np.argmax(Q[f, c]))
113     return flechas
```

- **Línea 107:** recibe la libreta ya entrenada.
- **Línea 108:** crea un "diccionario" vacío: una lista donde a cada casilla se le asigna su mejor movimiento.
- **Líneas 109–110:** recorre todas las casillas del laberinto.
- **Línea 111:** solo le interesan las casillas libres (`.`) y el inicio (`S`); las paredes y la meta no necesitan flecha.
- **Línea 112:** `argmax` busca cuál de los 4 movimientos tiene el puntaje más alto en esa casilla y lo guarda (0, 1, 2 o 3). Con la lista `FLECHAS` se puede convertir en ↑ ↓ ← →.
- **Línea 113:** entrega ese mapa. Es como el "manual" que aprendió el ratón: "si estás aquí, ve para allá".

## 13. Función `recorrer`: el examen final

```
116 def recorrer(Q):
117     estado = buscar(LABERINTO, "S")
118     camino = [estado]
119     for _ in range(MAX_PASOS):
120         accion = int(np.argmax(Q[estado[0], estado[1]]))
121         estado, _, termino, choco = paso(LABERINTO, estado, accion)
122         camino.append(estado)
123         if termino:
124             return camino, True
125         if choco:
126             return camino, False
127     return camino, False
```

- **Línea 116:** pone a prueba al ratón ya entrenado.
- **Línea 117:** lo coloca en el inicio.
- **Línea 118:** empieza a anotar el camino que recorre, comenzando por la casilla de inicio.
- **Línea 119:** le da hasta 40 pasos.
- **Línea 120:** esta vez **no hay nada al azar**: siempre elige el movimiento con mejor puntaje en su libreta.
- **Línea 121:** hace el movimiento. Los puntos no importan aquí, por eso se guardan en `_` (se ignoran).
- **Línea 122:** anota la casilla nueva en el camino.
- **Líneas 123–124:** si llegó a la meta, entrega el camino y "Sí llegó" (`True`).
- **Líneas 125–126:** si chocó con una pared, se detiene y entrega "No llegó". Se corta porque, si no hay azar, chocaría una y otra vez en el mismo lugar para siempre.
- **Línea 127:** si se acabaron los 40 pasos sin llegar, también entrega "No llegó".

## 14. El programa principal

```
130 if __name__ == "__main__":
131     Q, exitos, puntos = entrenar()
132     camino, llego = recorrer(Q)
133     print("Llego a la meta", sum(exitos), "de", len(exitos), "veces")
134     print("Puntos en el ultimo episodio:", puntos[-1])
135     print("Exito ultimos 20:", tasa_exito(exitos)[-1], "%")
136     print("Agente entrenado llega:", llego, "en", len(camino) - 1, "pasos")
```

- **Línea 130:** significa "haz lo de abajo **solo si este archivo se ejecuta directamente**". Si otro archivo (por ejemplo una app con gráficas) hace `import modelo` para usar sus funciones, esta parte no se ejecuta sola.
- **Línea 131:** entrena al ratón con los valores por defecto (300 intentos) y guarda la libreta, los éxitos y los puntos.
- **Línea 132:** hace el examen final y guarda el camino y si llegó o no.
- **Línea 133:** muestra cuántas veces llegó a la meta de los 300 intentos (`sum` suma todos los unos).
- **Línea 134:** muestra los puntos del último intento (`[-1]` significa "el último de la lista").
- **Línea 135:** muestra el porcentaje de éxito en los últimos 20 intentos.
- **Línea 136:** muestra si el ratón entrenado llegó a la meta en el examen y en cuántos pasos. Se resta 1 porque el camino incluye la casilla de inicio, que no es un paso.

## Resumen

1. Se dibuja un laberinto y se definen los movimientos y los puntos.
2. El ratón empieza sin saber nada (libreta llena de ceros).
3. Intenta salir 300 veces. En cada paso elige un movimiento, recibe puntos y corrige su libreta.
4. Con el tiempo, el premio de la meta se "contagia" hacia atrás y la libreta aprende cuál es el mejor movimiento en cada casilla.
5. Al final se le hace un examen sin azar para ver si ya sabe llegar solo, y se muestran los resultados.